

Adaptive Auto-Scaling with Dynamic Step Size Using AWS Lambda and AWS CloudWatch

Cloud Computing Presentation

Prof. Emiliano Casalicchio

Valerio Mesiti – Gabriele Moretti – Niccolo Siciliano

Academic Year 2024–2025



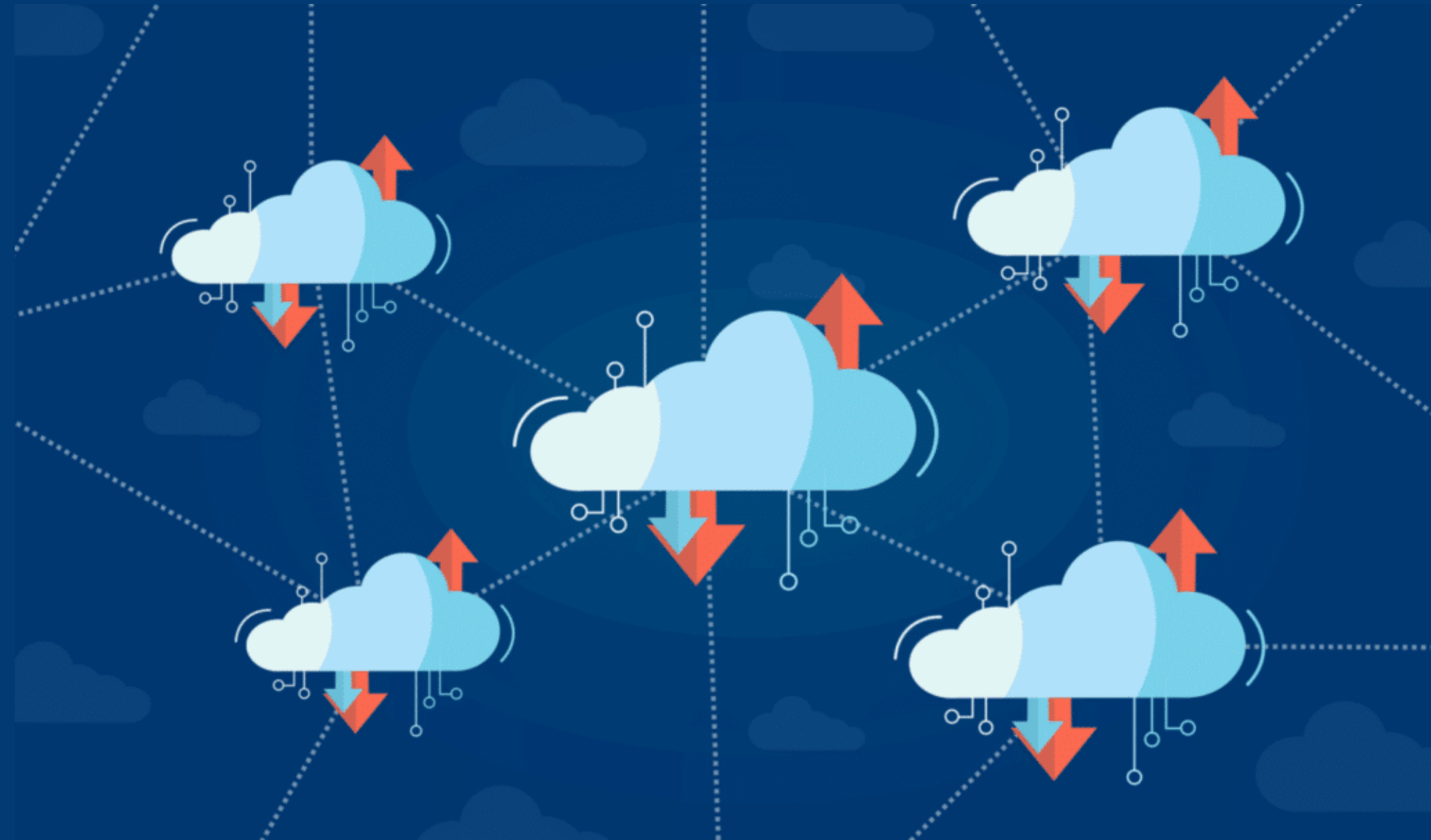
SAPIENZA
UNIVERSITÀ DI ROMA

Faculty of Information Engineering, Computer Science and Statistics
Department of Computer Science

We implemented an **adaptive auto-scaling strategy** for cloud environments, leveraging AWS Lambda and Amazon CloudWatch to dynamically scale EC2 instances in proportion to deviations from a target CPU utilization range

Performance evaluation was carried out using **three realistic workload** patterns: **bursty, clustered, and peaky**

Results show that the adaptive step sizing mechanism effectively **balances scalability and resource efficiency** under variable load conditions



Index

1. Introduction and Contextualization



- Introduction and Contextualization
- Architecture and Implementation
- Testing Methodology
- Results and Conclusion

General Overview

1. Introduction and Contextualization

Scalability is a **key challenge** in cloud environments, requiring systems to **adapt resources dynamically** to workload variations

We implemented an **adaptive auto-scaling algorithm**, inspired by Netto et al., to adjust the number of EC2 instances based on **real-time demand**

The solution integrates **AWS Lambda and Amazon CloudWatch** to ensure responsive scaling and efficient resource utilization



Evaluating Auto-scaling Strategies for Cloud Computing Environments

Marco A. S. Netto, Carlos Cardonha, Renato L. F. Cunha
IBM Research Brazil

Marcos D. Assunção
INRIA, LIP, ENS de Lyon, France

Abstract—Auto-scaling is a key feature in clouds responsible for adjusting the number of available resources to meet service demand. Resource pool modifications are necessary to keep performance indicators, such as utilisation level, between user-defined lower and upper bounds. Auto-scaling strategies that are not properly configured according to user workload characteristics may lead to unacceptable QoS and large resource waste. As a consequence, there is a need for a deeper understanding of auto-scaling strategies and how they should be configured to minimise these problems. In this work, we evaluate various auto-scaling strategies using log traces from a production Google data centre cluster comprising millions of jobs. Using utilisation level as performance indicator, our results show that proper management of auto-scaling parameters reduces the difference between the target utilisation interval and the actual values—we define such difference as Auto-scaling Demand Index. We also present a set of lessons from this study to help cloud providers build recommender systems for auto-scaling operations.

I. INTRODUCTION

Cloud infrastructure is typically elastic, *i.e.* the number of allocated resources can be modified dynamically according to users' needs. Typically, users define an upper bound U and a lower bound L on a target performance metric (*e.g.*, utilisation level, throughput, average response time) to trigger the activation and deactivation, respectively, of a certain number s of resources (step size). Unfortunately, users typically set U , L , and s in an ad-hoc manner, and, as a consequence, performance indicators of cloud solutions are frequently out of their target intervals. In the case of utilisation level, these deviations lead to unsatisfactory QoS and/or use of resources, so they are clearly undesired and should be minimised.

In addition to the challenges involved in the choice of parameters U , L , and s , one must also be careful when selecting an auto-scaling triggering strategy. Several mechanisms addressing this issue rely on forecasts of future resource usage in order to allocate and release resources pro-actively, but there are challenges to these approaches, such as:

- Inherent difficulties in load estimations—especially for environments with heterogeneous workloads;
- The relationship between the monitoring frequency and the time required by the cloud infrastructure to allocate and deallocate resources has a strong impact on the performance of reactive and predictive solutions.

In this article, we discuss and analyse auto-scaling solutions,

which vary according to the chosen strategies for triggering auto-scaling operations and for setting step sizes. We performed computational experiments on real-world data to show the impact of these choices on infrastructure utilisation. In particular, we evaluate the differences between the target interval and the actual measured values and investigate the root causes of undesired deviations. The key contributions of this paper are:

- **Auto-scaling Demand Index (ADI)**: a new performance metric for the evaluation of auto-scaling strategies that penalises differences between actual and desired resource utilisation levels (§ II);
- **Adaptive**: a strategy for defining adequate step sizes s in auto-scaling operations based on L , U , and the current system utilisation level (§ III);
- An extensive performance study that compares several auto-scaling strategies based on the ADI metric. The study utilises trace logs from a production data centre cluster from Google comprising millions of jobs and considers workloads with various characteristics (§ IV).

II. PROBLEM DESCRIPTION

Cloud users allocate a pool of computing resources to provide services with a certain *QoS* while respecting *budget* restrictions. There is typically a maximum number m of resources they are willing to allocate, but the current number of active elements is ideally kept to a minimum. One performance indicator that takes these two conflicting objectives into account is *resource utilisation level*, which is the ratio between what is currently in use and what is available from the resource pool. Users typically employ rule-based systems that periodically monitor utilisation and trigger auto-scaling operations whenever it is out of a given target interval. Namely, if utilisation goes above an upper bound U , additional resources are allocated to improve QoS, and if it goes below a lower bound L , the resource pool is shrunk to reduce costs.

The aim of this work is to address the following question: “*how to keep resource utilisation inside a target interval*”. To answer this question, we introduce in this article a metric called **Auto-scaling Demand Index (ADI)**, which is the sum of all distances computed between each utilisation level reported by the system and the target utilisation interval set by

Research Goal

1. Introduction and Contextualization

Our goal is to design a system that **dynamically adapts computing resources to workload variations**

The objective is to **prevent** both overload and waste, ensuring efficient resource utilization

We aim to **evaluate** whether this adaptive mechanism can keep performance within stable and desired thresholds



Index

2. Architecture and Implementation

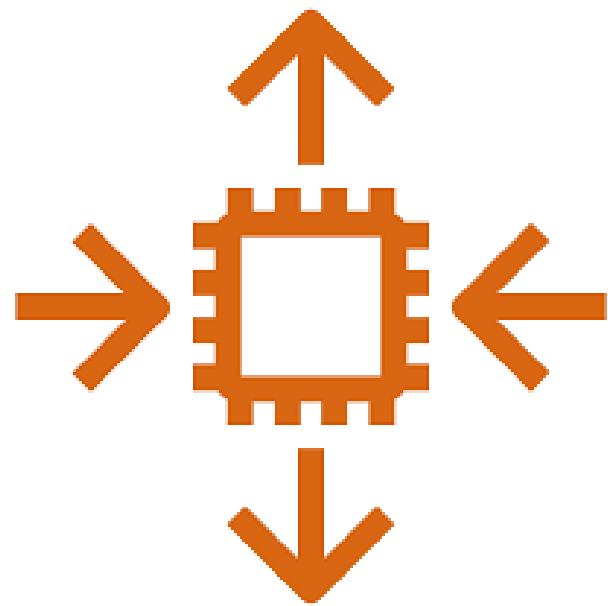


- Introduction and Contextualization
- Architecture and Implementation
- Testing Methodology
- Results and Conclusion

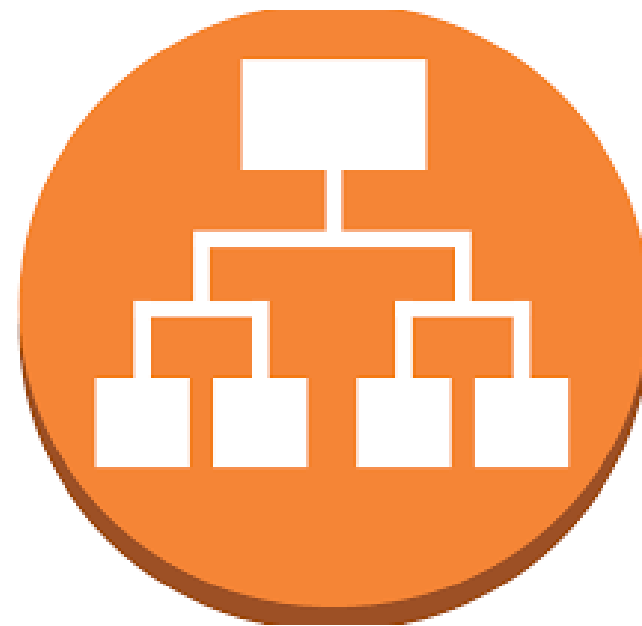


System Components

2. Architecture and Implementation



Auto Scaling Group (ASG)
automatically manages the
number of EC2 instances



Application Load Balancer
distributes incoming
requests evenly among the
healthy instances of the ASG



Lambda Function
implements the adaptive
scaling algorithm in Python



CloudWatch
monitors the system by
collecting detailed metrics

Functioning of the Algorithm



2. Architecture and Implementation

The algorithm aims to keep CPU utilization within the target range of **[45%, 80%]** by dynamically adjusting the number of active instances

The function reads the average CPU utilization for **all active instances** in the ASG using Amazon CloudWatch. Depending on whether the current use is:

- Inside the interval [45%, 80%], **no scaling is performed**
- Greater than 80%, a **scale-out** (addition of instances) is performed.
- Lower than 45%, a **scale-in** (removal of instances) is performed.

Changes to the Auto Scaling Group are **applied immediately**, ensuring responsiveness while minimizing unnecessary actions and costs

- For scale-out
 - Lower bound $L_t = m_t \times [(u_t - U) / U]$
 - Upper bound $U_t = m_t \times [(u_t - L) / L]$
 - Step size $s_t = \alpha \times L_t + (1 - \alpha) \times U_t$
- For scale-in
 - Lower bound $L_t = m_t \times [(U - u_t) / U]$
 - Upper bound $U_t = m_t \times [(L - u_t) / L]$
 - Step size $s_t = (1 - \alpha) \times L_t + \alpha \times U_t$

Aggressiveness parameter
 $\alpha = 0.5$

Index

3. Testing Methodology



- Introduction and Contextualization
- Architecture and Implementation
- Testing Methodology
- Results and Conclusion



Testing Methodology

3. Testing Methodology

The goal of the testing phase was to evaluate the dynamic behavior of the system under **varying workload conditions** and verify whether the adaptive algorithm can **effectively adjust the number of EC2 instances** based on CPU Utilization, maintaining performance close to a predefined target

Workloads were generated with the **stress tool**, automatically launched on each EC2 instance via a **user data script**

We performed a total of **three experiments**, varying the intensity and duration of the load to evaluate the reactivity and stability of the control mechanism



Evaluated Cloud Features

3. Testing Methodology

The evaluation targets two key aspects of AWS auto-scaling services:

1. Behavior of the Auto Scaling Group (ASG)

- Responsiveness to changes in load
- Consistency of active instances with the algorithm's dynamic step size
- Avoidance of oscillations or unstable scaling

2. Performance of EC2 instances

- Average CPU utilization, to ensure it stays within the target range $[L, U]$
- Load distribution, to verify correct balancing by the ALB and ASG
- Launch and “healthy” time for new instances, as an indicator of auto-provisioning effectiveness

Since the algorithm employs a variable strategy, the evaluation focuses on how the **step size adapts to the load**, whether scaling is more aggressive under high demand and gentler under light loads, and the impact of the **aggressiveness parameter** (α) on system behavior



Performance Metrics

3. Testing Methodology

Metrics, drawn from AWS standard namespaces, were selected to **objectively assess the system's behavior over time**, in relation to throughput, resource usage, scalability, and availability. The metrics used are:

- **AWS/EC2 → CPUUtilization**: measures each instance's CPU usage, indicating computational load and serving as the benchmark for the algorithm's scaling decisions
- **AWS/AutoScaling → GroupDesiredCapacity**: tracks the number of EC2 instances the Auto-Scaling group aims to keep active, serving as the key metric for analyzing the dynamic scalability



Evaluation Tools

3. Testing Methodology

The project infrastructure uses EC2 instances managed by an **Auto Scaling Group** and balanced via an **Application Load Balancer**.

This setup allows the use of integrated AWS tools to monitor **key metrics** (throughput, availability, scalability, and resource usage), providing a system-oriented assessment of the auto-scaling mechanism. Specifically:

- **Amazon CloudWatch:** the primary AWS monitoring tool, collecting metrics from the ALB, EC2 instances, and the ASG to provide detailed system performance insights
- **User-data scripts on EC2 instances:** each instance runs a simple HTTP server that accepts parameters for *stress -ng*, enabling controlled, repeatable CPU load tests across instances

Experiment Design

3. Testing Methodology



The experiments send **HTTP POST requests** to the ALB's */start -stress* endpoint, triggering *stress -ng* on EC2 instances to generate CPU load based on specified parameters (CPUs, load percentage, timeout)

System behavior was tested with three workload profiles (bursty, clustered, and peaky) implemented via **bash scripts** controlling request frequency, payload, and cooldowns. While differing in **intensity and timing**, **all profiles aim** to assess ASG elasticity under sudden or irregular CPU demands

Each workload experiment lasted **3–4 hours with 225 requests**, during which the system remained fully operational and monitored via CloudWatch, under a constant environment (same VPC, EC2 instance type, scaling policies, and metric collection methods)

Workload Design

3. Testing Methodology



Workload	Pattern	CPU Load (stress -ng params)	Request Phases	Purpose
Bursty	Alternating warmup, sudden bursts, cooldowns	1-2 CPUs, 30-100% load, 200-250s	Multiple bursts separated by cooldown intervals	Simulates unpredictable traffic spikes (e.g., flash crowds)
Clustered	Large clusters of requests with cooldowns in between	1-2 CPUs, 50-100% load, 200-250s	3-4 main clusters, separated by long cooldowns	Represents batch-like workloads or synchronized user actions
Peak	Sharp peaks of increasing intensity	1-2 CPUs, 60-100% load, 150-200s	Progressive peaks with growing stress intensity	Tests system elasticity under extreme short-lived surges



Experimental Setup

3. Testing Methodology

The experimental environment, deployed in **AWS Academy Learner Lab** within a **single VPC**, includes:

- **Application Load Balancer** for request distribution
- **Auto Scaling Group** for instance management based on CPU usage
- **Launch Template** for instance configuration
- **Lambda function** triggered by **CloudWatch Events** to apply adaptive scaling logic

All resources were placed in the **same availability zone** to **minimize latency** and **ensure consistency** in testing

The environment remained **unchanged** during the experiments, so variations reflect only load dynamics, not infrastructural changes

Index

4. Results and Conclusion



- Introduction and Contextualization
- Architecture and Implementation
- Testing Methodology
- Results and Conclusion

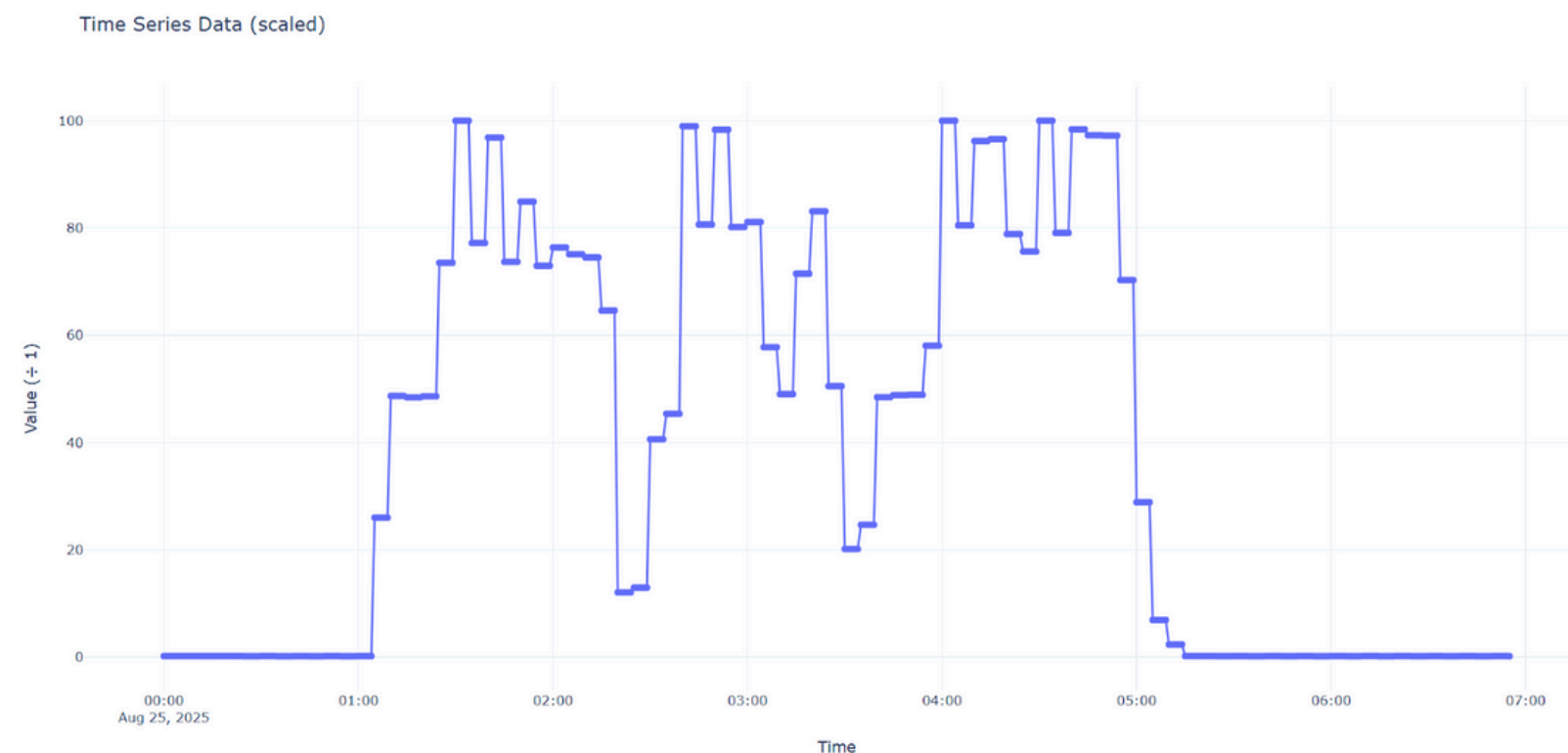
Workload 1 (Bursty) Results



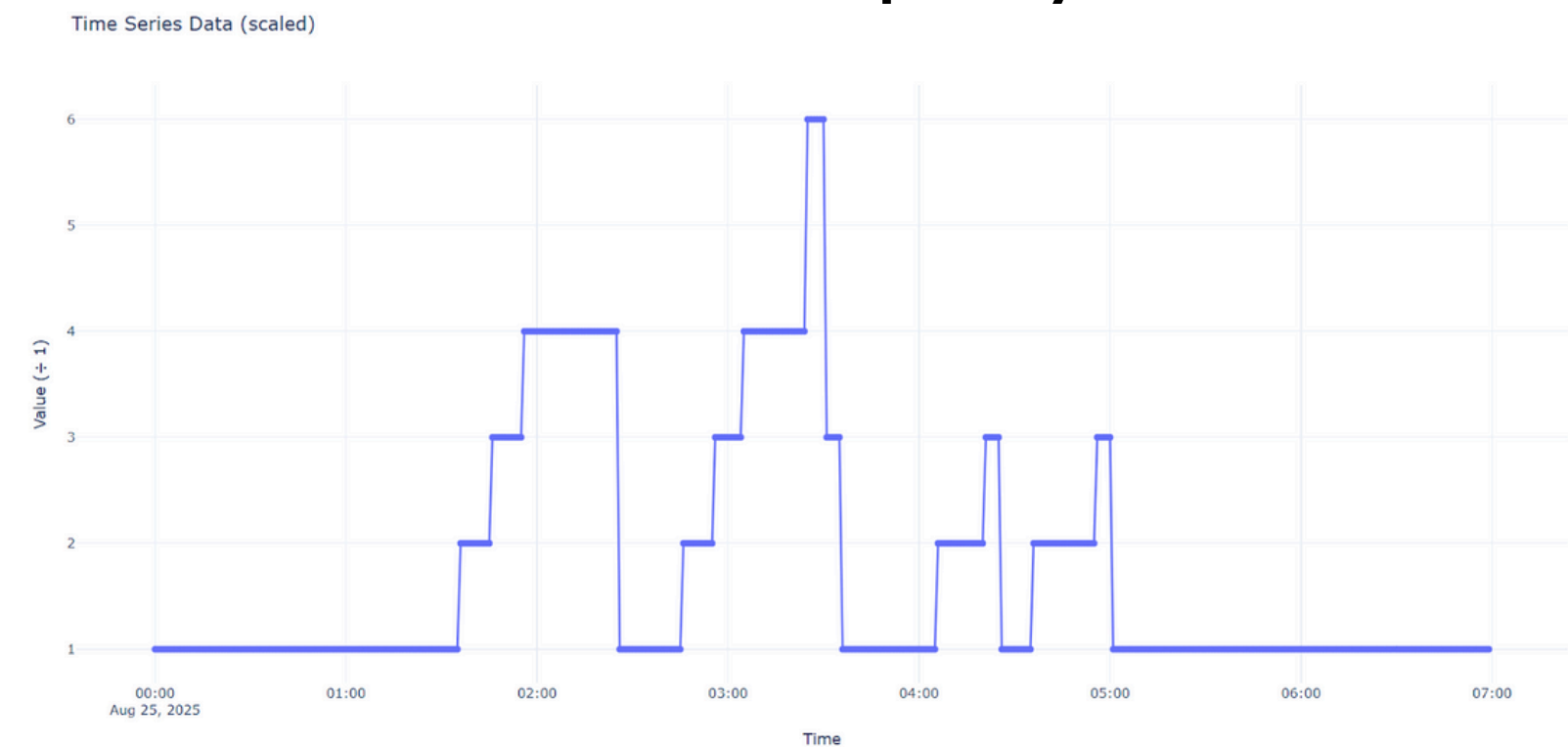
4. Results and Conclusion

- **CPUUtilization** graph shows rapid spikes up to 100%, demonstrating responsiveness to sudden load changes and stable behavior across repeated bursts
- **Desired Capacity** graph shows four stable scaling episodes, with the system dynamically adjusting from 1 to 6 instances in response to bursty load while optimizing costs during idle phases
- **ADI** = 9.524011200544498

CPUUtilization



Desired Capacity



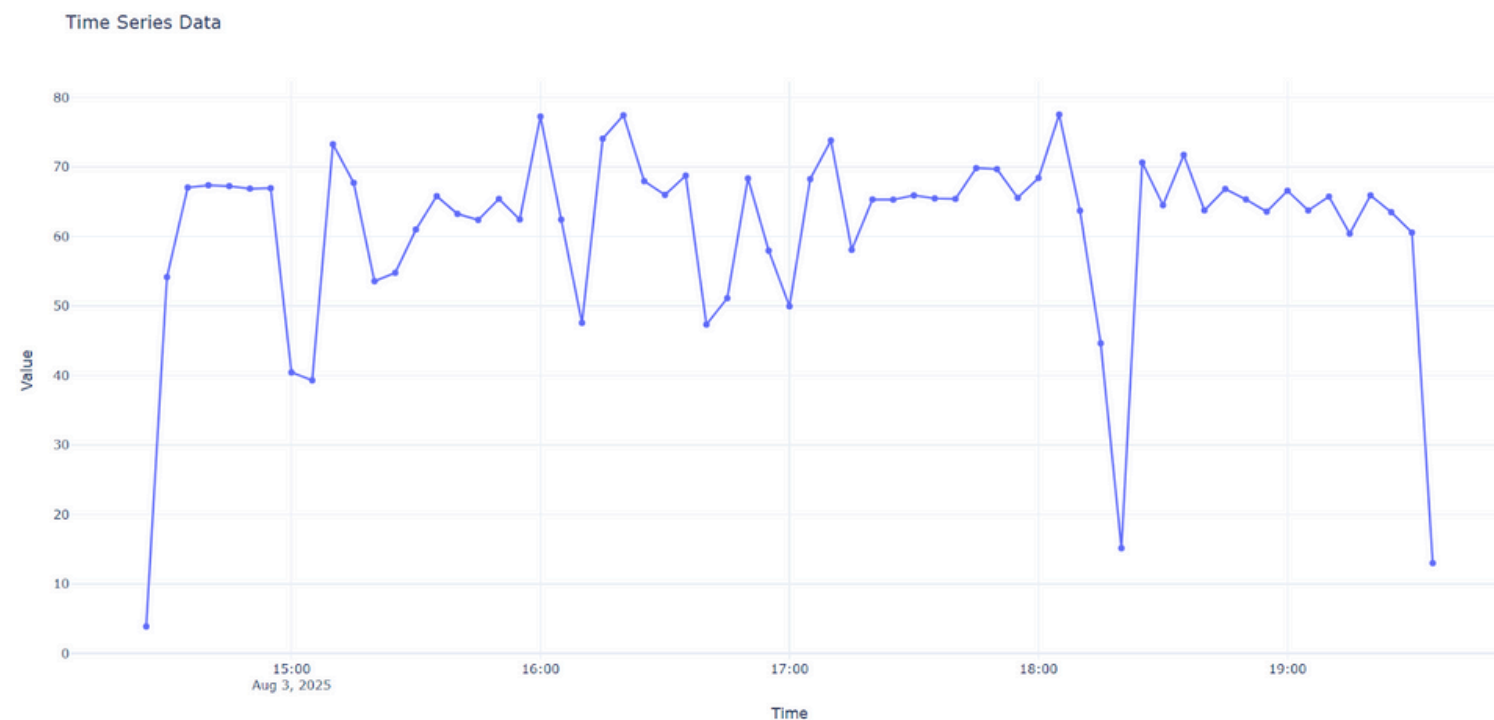
Workload 5 (Clustered) Results



4. Results and Conclusion

- **CPUUtilization** graph shows overall stable performance, with occasional sharp drops likely due to throttling or resource saturation
- **Desired Capacity** graph highlights gradual scaling with prolonged stable levels, reflecting efficient and calibrated management of clustered load
- **ADI** = 5.8034646864472008

CPUUtilization



Desired Capacity

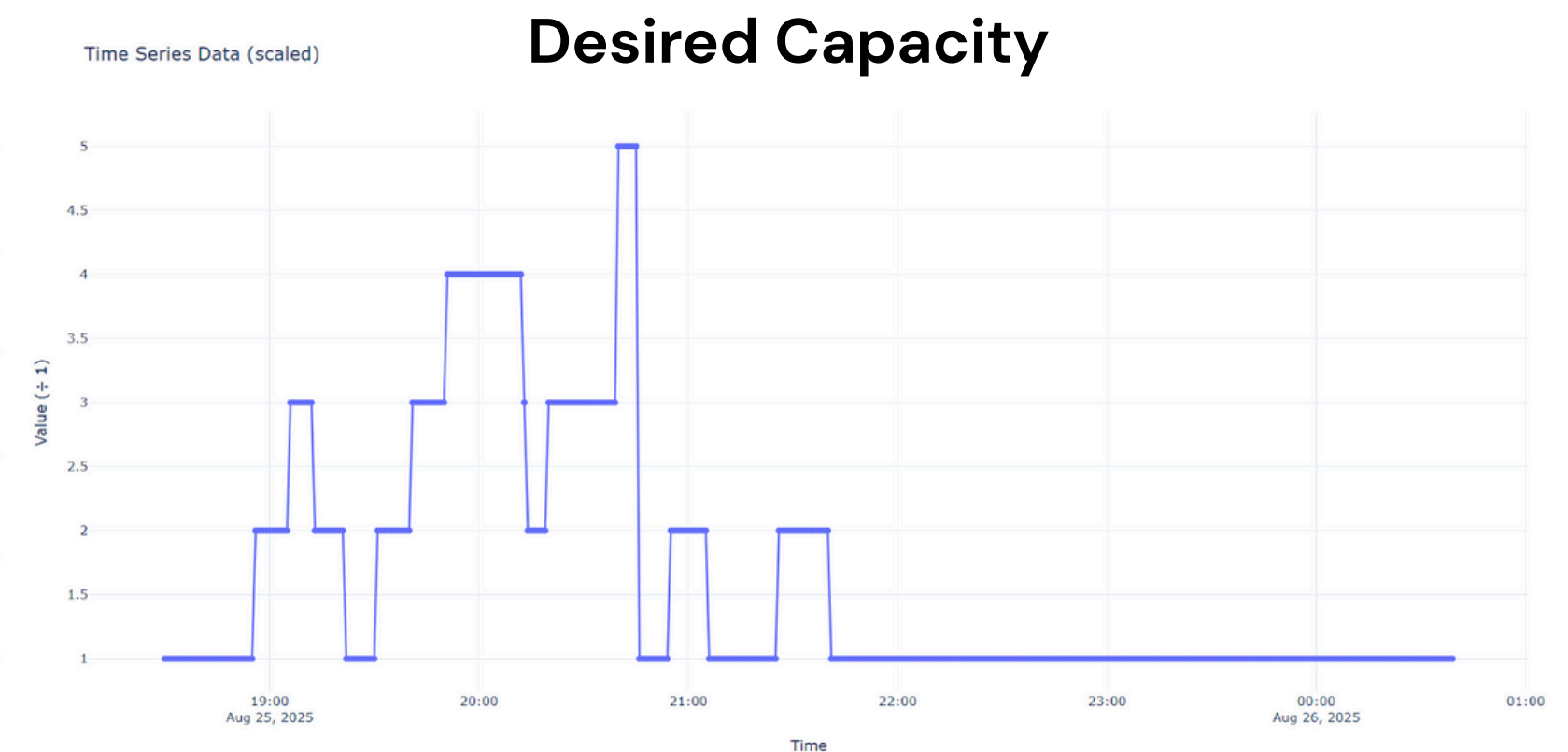
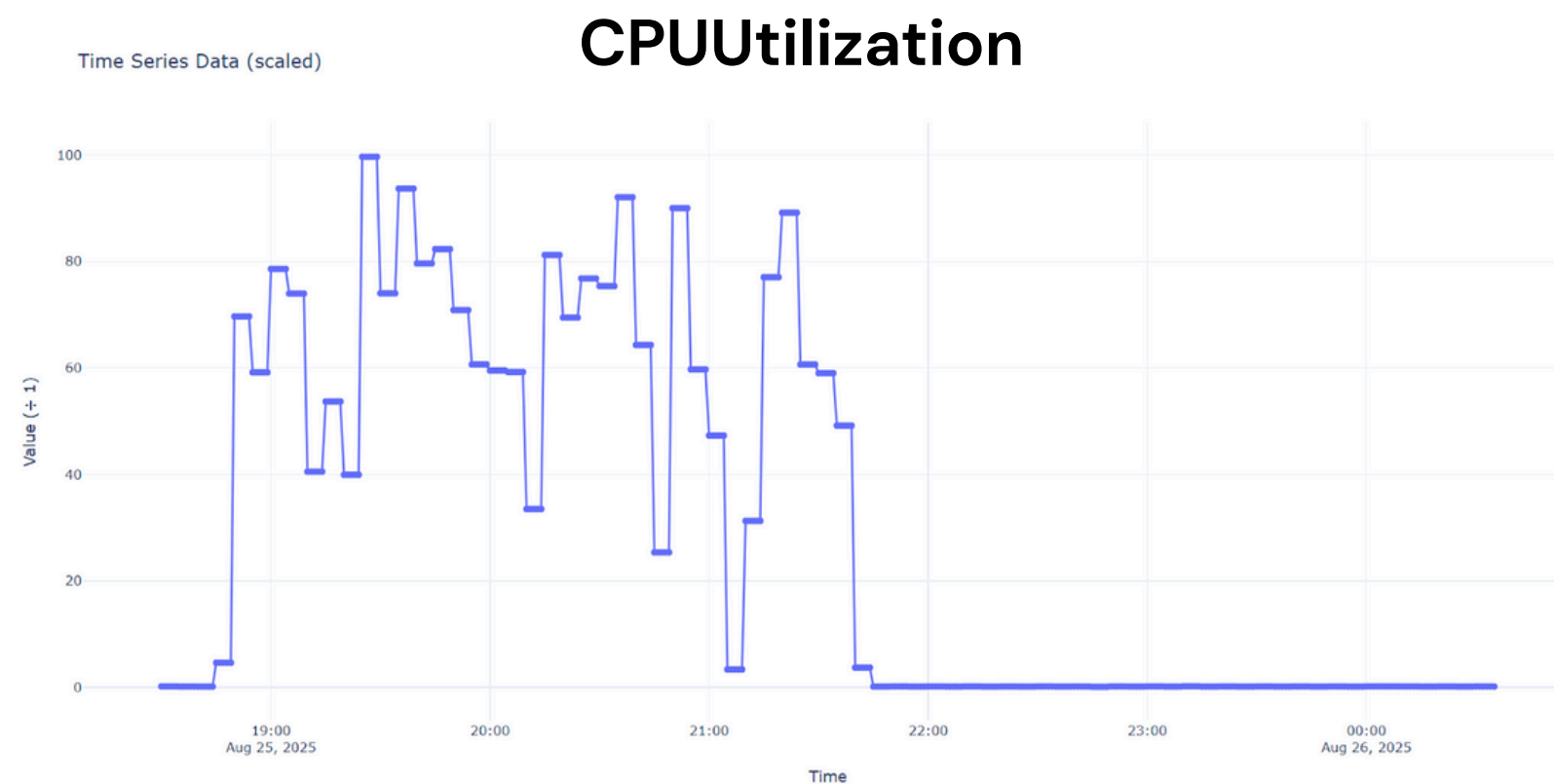


Workload 6 (Peaky) Results



4. Results and Conclusion

- **CPUUtilization** shows sharp spikes near 100%, reflecting irregular demand and strong system responsiveness
- **Desired Capacity** dynamically scales from 1 to 5 instances, growing stepwise during peaks and returning to the minimum when the load decreases, demonstrating effective auto-scaling
- **ADI** = 6.830014450684615



Conclusion

4. Results and Conclusion



The adaptive step-size algorithm **ensured proportional scaling**, keeping CPU utilization within the target range

The system demonstrated **responsiveness to variable and fast-changing workloads**, avoiding instability

Compared to static strategies, the adaptive approach **reduced both over- and under-provisioning, optimizing resource usage**

Overall, the results confirm that adaptive auto-scaling enhances **efficiency, reliability, and scalability in cloud environments**, representing a **valid solution** for dynamic environments with high load variability

Adaptive Auto-Scaling with Dynamic Step Size Using AWS Lambda and AWS CloudWatch

Cloud Computing Presentation

Prof. Emiliano Casalicchio

Valerio Mesiti – Gabriele Moretti – Niccolo Siciliano

Academic Year 2024-2025

Thank you for your attention! Questions?



SAPIENZA
UNIVERSITÀ DI ROMA

Faculty of Information Engineering, Computer Science and Statistics
Department of Computer Science